

Лабораторная работа № 1.2. «Лексический анализатор на основе регулярных выражений»

16 марта 2026 г.

Ольга Ивенкова, ИУ9-62Б

Цель работы

Целью данной работы является приобретение навыка разработки простейших лексических анализаторов, работающих на основе поиска в тексте по образцу, заданному регулярным выражением.

Индивидуальный вариант

Строковые литералы: ограничены обратными кавычками, могут занимать несколько строчек текста, для включения обратной кавычки она удваивается.

Числовые литералы: десятичные литералы представляют собой последовательности десятичных цифр, двоичные — последовательности нулей и единиц, оканчивающиеся буквой «b».

Идентификаторы: последовательности десятичных цифр и знаков «?», «*» и «|», не начинающиеся с цифры.

Лексический домен для защиты

Знаки операций отношения: >=, <=, >, <

Реализация

```
import java.util.regex.Pattern;
import java.util.regex.Matcher;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.nio.charset.StandardCharsets;
import java.io.IOException;
```

```

public class Lexer {
    public enum Tag {
        STRING, NUMBER_BIN, NUMBER_DEC, IDENT, RELATION
    }

    public static class Token {
        public final Tag tag;
        public final int line, pos;
        public final Object value;

        public Token(Tag tag, int line, int pos, Object value) {
            this.tag = tag;
            this.line = line;
            this.pos = pos;
            this.value = value;
        }
    }

    // Знаки операций отношения: >=, <=, >, <
    private static final Pattern[] PATTERNS = {
        Pattern.compile("\\A`([^\`]|``)*`"),
        Pattern.compile("\\A[01]+b"),
        Pattern.compile("\\A[0-9]+"),
        Pattern.compile("\\A[<>][=]?"),
        Pattern.compile("\\A[?*][0-9?*]*")
    };
    private static final Tag[] TAGS = {
        Tag.STRING, Tag.NUMBER_BIN, Tag.NUMBER_DEC, Tag.RELATION, Tag.IDENT
    };
    private static final int DOMAIN_COUNT = PATTERNS.length;

    private final String program;
    private int idx;
    private int line, pos;

    public Lexer(String program) {
        this.program = program;
        this.idx = 0;
        this.line = 1;
        this.pos = 1;
    }

    private void nextChar() {
        if (idx >= program.length()) return;
        char c = program.charAt(idx);
        if (c == '\n') {

```

```

        line++;
        pos = 1;
    } else {
        pos++;
    }
    idx++;
}

private void skipWhitespace() {
    while (idx < program.length() && Character.isWhitespace(program.charAt(idx))) {
        nextChar();
    }
}

private boolean isStartOfToken(char c) {
    return c == '>' || c == '<' || c == '`' || c == '?'
        || c == '*' || c == '|' || (c >= '0' && c <= '9');
}

public Token nextToken() {
    while (true) {
        skipWhitespace();
        if (idx >= program.length()) return null;

        int startLine = line;
        int startPos = pos;
        String remaining = program.substring(idx);

        int bestLen = -1;
        Tag bestTag = null;
        String bestLexeme = null;

        for (int i = 0; i < DOMAIN_COUNT; i++) {
            Matcher m = PATTERNS[i].matcher(remaining);
            if (m.find()) {
                int len = m.group().length();
                if (len > bestLen) {
                    bestLen = len;
                    bestTag = TAGS[i];
                    bestLexeme = m.group();
                }
            }
        }

        if (bestLen == -1) {
            System.out.println("syntax error (" + line + ", " + pos + ")");

```

```

        nextChar();
        while (idx < program.length() &&
            !isStartOfToken(program.charAt(idx))) {
            nextChar();
        }
        continue;
    }

    for (int i = 0; i < bestLen; i++) nextChar();

    return new Token(bestTag, startLine, startPos, bestLexeme);
}

}

public static void main(String[] args) throws IOException {
    if (args.length < 1) {
        System.err.println("Usage: java Lexer <filename>");
        return;
    }
    byte[] bytes = Files.readAllBytes(Paths.get(args[0]));
    String content = new String(bytes, StandardCharsets.UTF_8);
    Lexer lexer = new Lexer(content);
    Token t;
    while ((t = lexer.nextToken()) != null) {
        String tagName;
        switch (t.tag) {
            case STRING:           tagName = "STRING"; break;
            case NUMBER_BIN:       tagName = "NUMBER"; break;
            case NUMBER_DEC:       tagName = "NUMBER"; break;
            case RELATION:         tagName = "RELATION"; break;
            default:               tagName = "IDENT";
        }
        System.out.println(tagName + " (" + t.line + ", "
            + t.pos + "): " + t.value);
    }
}
}

```

Тестирование

Входные данные

`Hello, world

дльдль

лдтлю!` 01b 42 ?var <= 78b ?78*?6

Вывод на stdout

```
STRING (1,1): `Hello, world
дльдль
лдтлю! `
NUMBER (3,9): 01b
NUMBER (3,13): 42
IDENT (3,16): ?
syntax error (3,17)
RELATION (3,22): <=
NUMBER (3,25): 78
syntax error (3,27)
IDENT (3,29): ?78*?6
```

Вывод

Приобрела навык разработки простейших лексических анализаторов, работающих на основе поиска в тексте по образцу, заданному регулярным выражением.